

Building a Secure Chat Application

Phase 0 — Deployment Environment

Phase 1 — Baseline Chat Application

Phase 2 — Confidentiality via Diffie–Hellman

Phase 3 — Server Authentication via PKI

Phase 4 — End-to-End Encryption Between Clients

Phase 5 — Forward Secrecy via Key Rotation

A one-to-one chat application over TCP sockets, hardened across five phases. This report covers the deployment environment, the baseline plaintext application, confidentiality via Diffie–Hellman, server authentication via a PKI, end-to-end encryption between the two clients, and forward secrecy via key rotation, each with its verification.

Four virtual machines were provisioned on an isolated virtual network and connectivity verified before any application code was run. The application is written in C++17 over POSIX sockets, with a length-prefixed record protocol chosen so that the framing remains valid once message bodies stop being printable text. Phase 1 deliberately provides no confidentiality: the server logs every message and a packet capture shows the whole conversation in cleartext. Phase 2 closes that gap with a Diffie–Hellman key exchange written from first principles and AES-256-GCM record encryption, then demonstrates a man-in-the-middle proxy defeating it. Phase 3 adds a Public Key Infrastructure — a Certificate Authority, a server certificate, and a proof of possession — so the client authenticates the server and the same attack now fails. Phase 4 adds encryption directly between the two clients, riding inside the relay’s message field, so the server forwards only opaque data it cannot read — with no change to the server at all. Phase 5 rotates that end-to-end key on a timer, each new key independent of the last, so a compromised key exposes only a bounded window. No TLS library is used anywhere.

Bathula Rithvik

23B0939

September 4, 2026

Contents

1	Introduction	2
2	Phase 0 — Deployment Environment	2
2.1	Requirement	2
2.2	Topology	2
2.3	Design notes	2
2.4	Connectivity verification	3
2.5	Toolchain	4
3	Phase 1 — Baseline Chat Application	4
3.1	Requirements	4
3.2	Protocol design	4
3.3	Implementation	6
3.4	Verification	7
3.5	Evidence index	9
3.6	Deliverables checklist	9
4	Phase 2 — Confidentiality via Diffie–Hellman	9
4.1	Requirements	9
4.2	Diffie–Hellman from first principles	10
4.3	Why the shared secret is hashed	10
4.4	Record encryption	10
4.5	Verification	10
4.6	Attack — man-in-the-middle	12
4.7	Deliverables checklist	12
5	Phase 3 — Server Authentication via PKI	12
5.1	Requirements	12
5.2	The PKI	13
5.3	Certificate and proof of possession	13
5.4	Client validation	13
5.5	Verification — legitimate flow	13
5.6	Attack — the MITM now fails	15
5.7	Deliverables checklist	15
6	Phase 4 — End-to-End Encryption Between Clients	15
6.1	Requirements	15
6.2	Design	16
6.3	Verification	16
6.4	Deliverables checklist	18
7	Phase 5 — Forward Secrecy via Key Rotation	18
7.1	Requirements	18
7.2	Design	18
7.3	Verification	18
7.4	What forward secrecy buys	19
7.5	Deliverables checklist	19
8	Building and Running	19

A Generative AI Usage**20**

1 Introduction

This report documents a one-to-one chat application built over TCP sockets and progressively hardened across five phases. It covers the deployment environment (Phase 0), the baseline plaintext chat application (Phase 1), confidentiality via Diffie–Hellman with a man-in-the-middle attack against it (Phase 2), server authentication via a PKI that defeats that attack (Phase 3), end-to-end encryption between the clients (Phase 4), and forward secrecy via key rotation (Phase 5), each with its verification.

The application is written in C++17 using POSIX sockets only. No TLS/SSL library is used anywhere: `<openssl/ssl.h>` and `<openssl/dh.h>` are excluded by design, as required. Phase 1 introduces no cryptography at all — its purpose is to establish the baseline against which every later phase is measured, and specifically to demonstrate that an unprotected chat protocol is fully readable both by the relaying server and by anyone capturing traffic on the network segment.

One decision shaped the implementation more than any other. The simplest framing that satisfies Phase 1 would have to be discarded as soon as message bodies stop being printable text, so a binary-safe format was adopted from the start instead. Section 3.2 sets out that choice and the reasoning behind it.

2 Phase 0 — Deployment Environment

2.1 Requirement

The assignment requires the application to be implemented and demonstrated across separate virtual machines rather than on a single machine over loopback, so that packet captures and later man-in-the-middle demonstrations are meaningful. The minimum topology is one server VM and two client VMs on a common virtual network; a fourth VM is required for the attack tasks in Phases 2 and 3.

2.2 Topology

Four Ubuntu 26.04.1 LTS Server virtual machines run under VirtualBox 7.2.16 on an Ubuntu 24.04 host. Each VM has two network adapters, serving distinct purposes:

- **Adapter 1 — NAT** (`enp0s3`). Provides outbound internet access for package installation, and inbound SSH from the host through a loopback-bound port forward. It is not part of the assignment network.
- **Adapter 2 — Internal Network** `secure-chat` (`enp0s8`). A private virtual Ethernet switch connecting only these four VMs. All chat traffic, all packet captures, and all later attack traffic occur here.

Role	VM	secure-chat IP	MAC (Adapter 2)	RAM	Purpose
S	vm1-server	10.10.0.10	08:00:27:11:08:0E	2048 MB	chat server
C1	vm2-client1	10.10.0.11	08:00:27:2A:3C:7B	1536 MB	client
C2	vm3-client2	10.10.0.12	08:00:27:32:22:C2	1536 MB	client
M	vm4-mallory	10.10.0.13	08:00:27:26:94:51	1536 MB	MITM proxy (Phases 2–3)

Table 1. Virtual machine roles and addressing on the `secure-chat` network (10.10.0.0/24).

2.3 Design notes

Figure 1 shows how the two adapters divide the work.

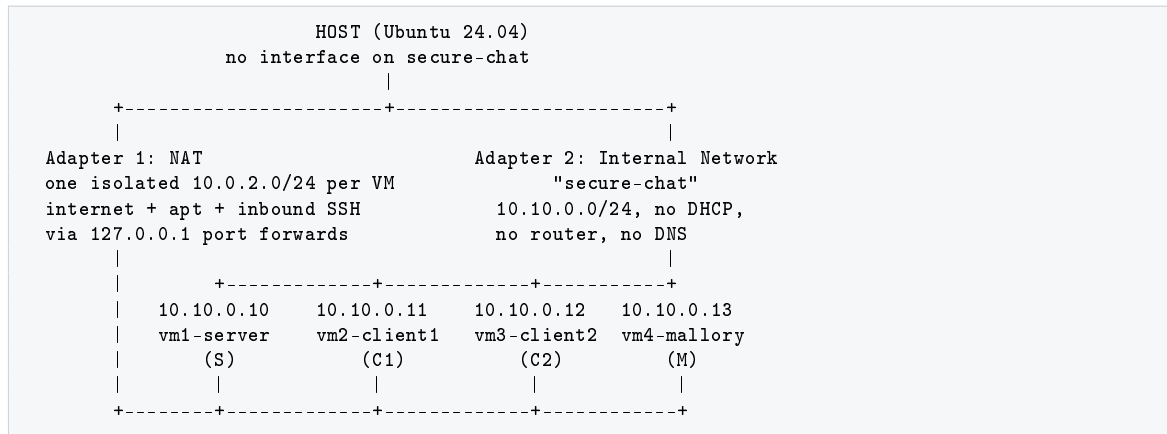


Figure 1. VM topology. The chat service listens on TCP port 5555 on 10.10.0.10.

Why two adapters. VirtualBox NAT creates a *separate, isolated* network per virtual machine rather than one shared network. Consequently every VM sees itself as 10.0.2.15, with gateway 10.0.2.2 — the identical addressing is correct, not a misconfiguration, because those addresses have meaning only inside each VM’s own NAT network. The direct consequence is that VMs cannot reach one another over NAT at all, which is precisely why the second adapter exists.

Why static addressing. A VirtualBox Internal Network is a bare virtual switch with no DHCP server, so `enp0s8` comes up with only an IPv6 link-local address and no IPv4. Addresses are therefore assigned statically through a dedicated netplan file, `/etc/netplan/99-secure-chat.yaml`, kept separate from the installer’s configuration so that an error can never break the interface carrying the SSH session:

```

network:
  version: 2
  ethernets:
    enp0s8:
      dhcp4: false
      addresses:
        - 10.10.0.10/24          # .11 on C1, .12 on C2, .13 on M

```

No default gateway and no nameservers are configured on this interface. The `secure-chat` network has no router, and a second default route would compete with the NAT adapter’s, potentially handing internet traffic to a network with nothing to forward it.

Capture vantage point. The host has no interface on `secure-chat` and therefore cannot observe chat traffic at all. All packet capture runs inside the VMs with `tshark -i enp0s8`; the resulting `.pcap` files are copied to the host for analysis in the Wireshark GUI.

2.4 Connectivity verification

The assignment requires basic connectivity between VMs to be verified with ping before the chat application is run. A full 4×4 matrix was executed, each VM pinging all four addresses:

=== from s ===	=== from c1 ===	=== from c2 ===	=== from m ===
10.10.0.10 OK	10.10.0.10 OK	10.10.0.10 OK	10.10.0.10 OK
10.10.0.11 OK	10.10.0.11 OK	10.10.0.11 OK	10.10.0.11 OK
10.10.0.12 OK	10.10.0.12 OK	10.10.0.12 OK	10.10.0.12 OK
10.10.0.13 OK	10.10.0.13 OK	10.10.0.13 OK	10.10.0.13 OK

All 16 checks succeeded. A representative single probe confirms the peers are directly on-link:

```
$ ping -c1 -W1 10.10.0.11
64 bytes from 10.10.0.11: icmp_seq=1 ttl=64 time=0.295 ms
```

The TTL of 64 is unchanged from the Linux default, confirming zero router hops — as expected on a single switched segment. Layer-2 adjacency was confirmed independently: the MAC addresses resolved by ARP on the server match the VirtualBox adapter configuration exactly, so each address provably belongs to the intended virtual machine.

Full environment records are in `evidence/phase0/`.

2.5 Toolchain

Identical on all four VMs: g++ 15.2.0, OpenSSL 3.5.5 with `libssl-dev` headers (needed from Phase 2 for `bn.h`, `evp.h` and `x509.h`), and TShark 4.6.4. Binaries are compiled on the VMs rather than on the host, since the host runs OpenSSL 3.0.13.

3 Phase 1 — Baseline Chat Application

3.1 Requirements

Phase 1 requires a chat server supporting exactly one server and two clients connected simultaneously over TCP, with messages typed by C1 delivered to C2 through the server and vice versa, relayed as-is with no encryption, and the command interface implemented from this phase onward.

3.2 Protocol design

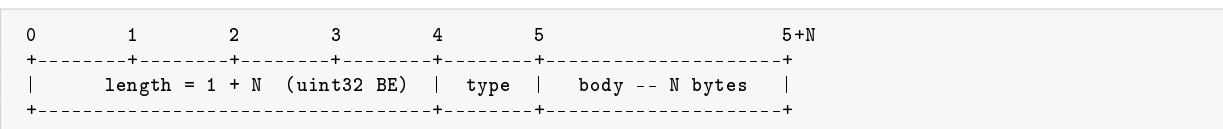
The protocol is split into two layers, so that neither has to change when the other does:

Layer	Responsibility
L2	Application grammar (LOGIN, MSG, WHO, QUIT)
L1	Record framing over the TCP byte stream

L1 knows nothing about chat; L2 knows nothing about how bytes are delimited, so later phases can insert processing between the two rather than modify either. The specification is in `docs/protocol.md`.

3.2.1 Message framing — deciding when a message is complete

TCP provides a reliable, ordered *byte stream*, not a message stream. A single `send()` may arrive split across several `recv()` calls, or concatenated with its neighbours. The protocol must therefore define its own message boundaries. Every record, in both directions and in every phase, is length-prefixed:



Reading a record is therefore a two-stage operation, each stage looping until satisfied: read exactly four bytes, decode the length with `ntohl()`, then read exactly that many further bytes. A message is complete when, and only when, the declared number of bytes has arrived.

Why not a newline delimiter. Delimiting messages with `\n` would be simpler, and entirely adequate for Phase 1 where every payload is printable text. It was rejected because the body will not stay printable: once payloads carry encrypted data they are uniformly distributed binary, so

approximately one byte in 256 equals 0x0A — a newline-framed reader would split messages at random positions. The available escapes are to Base64-encode every record (a 33 % overhead, and framing is still needed on top) or to introduce an escaping scheme (whose bugs would be protocol-breaking). Length prefixing is binary-safe, so the framing written now does not have to be revisited later.

This was verified empirically rather than assumed: a test transmitted a message containing all 256 possible byte values, including 0x00, 0x0A and 0xFF, and confirmed byte-exact delivery.

Denial-of-service guard. A declared length above 16384 is rejected and the connection closed *before any allocation*. A length field trusted blindly is a one-packet denial of service: a peer claiming 0xFFFFFFFF would otherwise trigger a 4 GB allocation.

Record type byte. The single type byte distinguishes application data (0x02, the only type Phase 1 emits) from handshake records (0x01) and plaintext alerts (0x03). Reserving the codes now means that adding record kinds later requires neither a version bump nor a change to existing parsers.

3.2.2 Message grammar

The body of an application record is a single UTF-8 line: a verb followed by space-separated arguments.

Client → Server	Server → Client
LOGIN <username>	OK <username>
MSG <to> <text>	ERR <reason>
WHO	FROM <sender> <text>
QUIT	USERS <name> <name> ...
	INFO <text>

LOGIN registers a username so the server can route by name. It carries no password: the assignment never requires user authentication, and its reference to protecting a login exchange is explicitly conditional. Being ordinary application data, this message will be covered automatically by any protection later applied to the record body.

3.2.3 Routing, and more than two clients

The server maintains a username → socket map. On receiving MSG <to> <text> it looks up the recipient, forwards FROM <sender> <text> to that socket, and replies ERR no such user: <to> if the recipient is absent. Because routing is a map lookup rather than a hard-wired pairing, the design already supports an arbitrary number of simultaneous clients; the two required by Phase 1 are simply the case that is demonstrated. Broadcast is used only for join and leave notices.

The opacity rule. The server parses exactly two tokens of a MSG: the verb and the recipient. The message text is copied byte-for-byte and never inspected, validated or modified. This is a deliberate constraint rather than an implementation detail: keeping the relay ignorant of message content means the routing logic never has to change when the content does.

3.2.4 Command interface

The commands required from Phase 1 are implemented in the client and map onto the wire protocol as follows. They are user-interface concepts, deliberately kept distinct from the protocol messages.

User types	Client action	Sent on the wire
@bob hello	select bob, send	MSG bob hello
@bob	select bob	—
/chat bob	select bob	— (local state only)
/who	request the user list	WHO
/quit	disconnect and exit	QUIT
anything else	send to the selected partner	MSG <partner> <text>

The final row follows the specification literally: input matching no recognised command tag is treated as a plain chat message to the currently selected user. A mistyped `/quti` is therefore transmitted as chat text rather than rejected.

3.3 Implementation

3.3.1 Server

Single-threaded and `poll()`-driven, with all sockets non-blocking. Each connection owns an inbound record reader that buffers partial records, and an outbound byte queue flushed when the socket reports itself writable.

This structure is what prevents one client from stalling the others. A blocking design would have to wait inside `recv()` for the remainder of a half-delivered record, or inside `send()` for a client that has stopped reading; in either case every other client waits too. With buffering on both directions, a partial record simply remains buffered until more data arrives, and a slow reader only accumulates its own backlog — capped, so that a peer which never reads is dropped rather than consuming server memory without limit.

```
RecordReader::Status RecordReader::next(uint8_t& type, std::vector<uint8_t>& body)
{
    size_t avail = buf_.size() - off_;
    if (avail < HDR_LEN) return Status::NeedMore;

    uint32_t len;
    std::memcpy(&len, buf_.data() + off_, HDR_LEN);
    len = ntohl(len);

    if (len < 1 || len > MAX_RECORD) return Status::Malformed;
    if (avail < HDR_LEN + len) return Status::NeedMore;
    ...
}
```

Listing 1. Incremental record extraction (`proto.cpp`).

3.3.2 Client

The client must wait on two sources at once: the socket and the keyboard. Blocking on either must not make it deaf to the other, so it uses two threads — the main thread reads standard input and parses commands, while a reader thread blocks on the socket and prints what arrives. The socket itself remains blocking, since each thread blocks on its own source. Terminal output is serialised behind a mutex so that an arriving message cannot interleave inside a line being printed.

3.3.3 Robustness

Both programs ignore `SIGPIPE`, so that writing to a socket whose peer has departed returns an error instead of terminating the process. Beyond the framing guard already described, the server rejects application data sent before `LOGIN`, duplicate usernames, usernames outside `[A-Za-z0-9_]`, oversized payloads and unexpected record types, closing the connection in each case.

3.4 Verification

3.4.1 Test setup

The server ran on `vm1-server`, with clients `alice` on `vm2-client1` and `bob` on `vm3-client2`. Packet capture ran simultaneously on all three VMs (`server.pcap`, `c1.pcap`, `c2.pcap`), filtered to TCP port 5555 on `enp0s8`.

3.4.2 Functional result

Both clients connected simultaneously and exchanged messages in both directions. The session exercised `@username` message, `/who`, `/chat`, implicit send to the selected partner, join and leave notices, and `/quit` on both ends.

3.4.3 The server can read every message

The server logs every message it relays, with the full text. The complete log for the demonstration session (`evidence/phase1/server.log`):

```
[2026-09-04T00:54:46Z] START phase1 plaintext relay on 0.0.0.0:5555 (log: server.log)
[2026-09-04T00:54:52Z] CONN 10.10.0.11:58172
[2026-09-04T00:54:52Z] LOGIN alice from 10.10.0.11:58172
[2026-09-04T00:54:52Z] CONN 10.10.0.12:42680
[2026-09-04T00:54:52Z] LOGIN bob from 10.10.0.12:42680
[2026-09-04T00:54:54Z] WHO  alice -> [alice bob]
[2026-09-04T00:54:56Z] RELAY alice -> bob : "hey bob, are you there?"
[2026-09-04T00:54:58Z] RELAY bob -> alice : "yes alice, receiving you loud and clear"
[2026-09-04T00:55:00Z] RELAY alice -> bob : "this line goes to the selected partner"
[2026-09-04T00:55:02Z] RELAY bob -> alice : "the server can read every word of this"
[2026-09-04T00:55:05Z] RELAY alice -> bob : "plaintext is fully readable on the wire"
[2026-09-04T00:55:08Z] WHO  bob -> [alice bob]
[2026-09-04T00:55:09Z] QUIT  alice
[2026-09-04T00:55:09Z] CLOSE alice (quit)
[2026-09-04T00:55:10Z] QUIT  bob
[2026-09-04T00:55:10Z] CLOSE bob (quit)
```

Every RELAY line carries the complete message text, printed by the server at the moment it forwards the message. **The server is a full plaintext observer of the conversation: it can read the entire content of every message passing through it.**

This log line is the anchor for the assignment as a whole. The same statement, unchanged, will serve in a later phase as evidence that the server can no longer read anything. Identical code, opposite conclusion.

3.4.4 Wireshark — plaintext on the wire

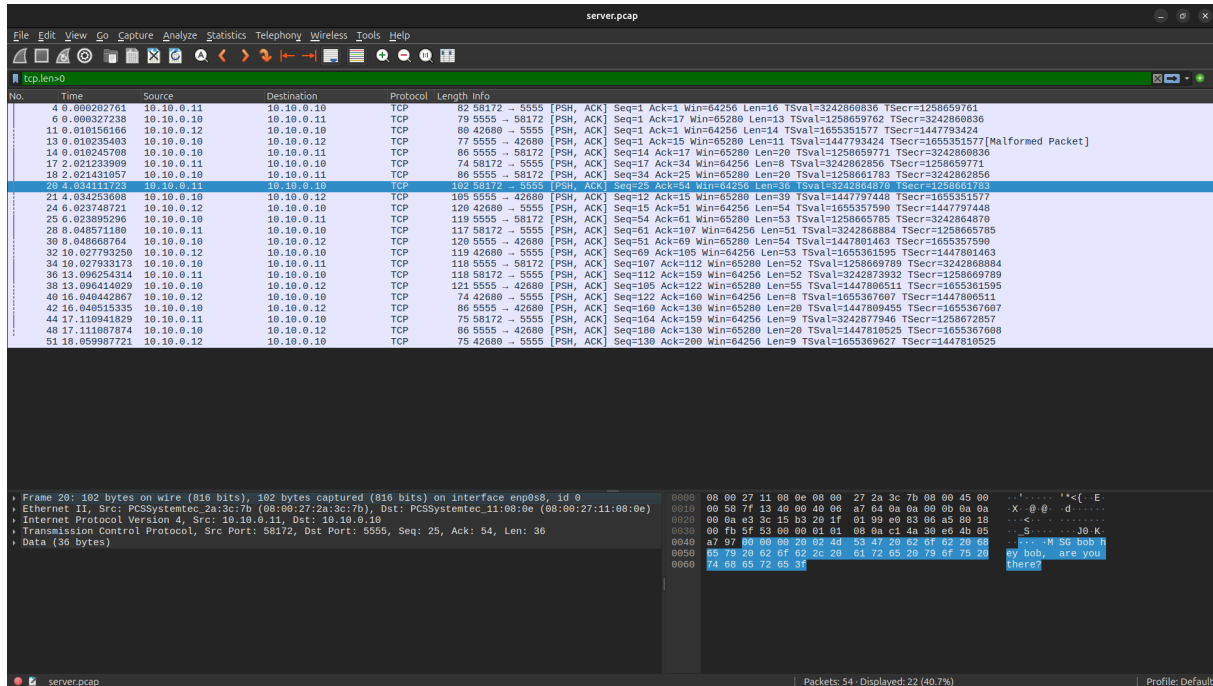


Figure 2. server.pcap opened in Wireshark, filtered to `tcp.len>0`. Frame 20 is selected: the bytes pane shows the payload `MSG bob hey bob, are you there?` directly readable in ASCII. The preceding five bytes are the record header — the 4-byte length prefix `00 00 00 20` (`32 = 1` type byte + `31` payload bytes) followed by the type byte `02`.

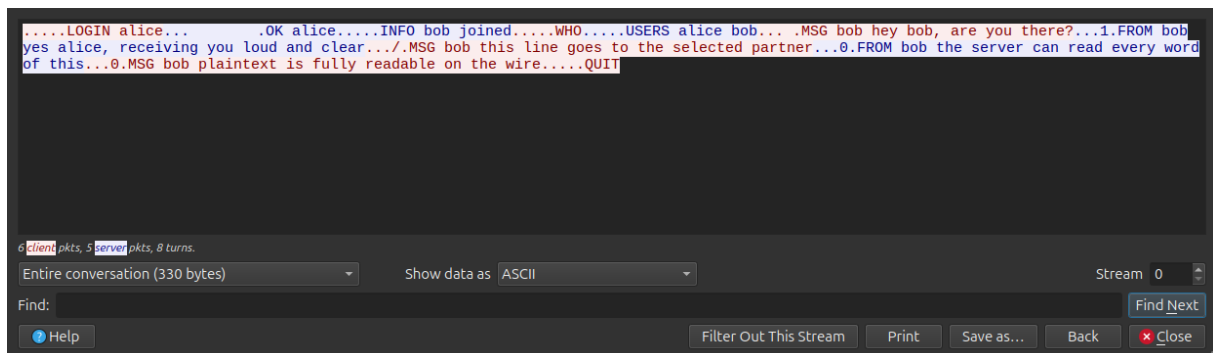


Figure 3. Follow → TCP Stream on server.pcap, stream 0 (client C1 ↔ server). The entire conversation is legible with no key and no decryption step: the login exchange, the user list, and every chat message in both directions. Client-to-server data is shown in red, server-to-client in blue.

The stream contents reproduce as:

```
....LOGIN alice...      .OK alice....INFO bob joined....WHO....USERS alice bob...
.MSG bob hey bob, are you there?...1.FROM bob yes alice, receiving you loud and clear
.../.MSG bob this line goes to the selected partner...0.FROM bob the server can read
every word of this...0.MSG bob plaintext is fully readable on the wire....QUIT
```

The non-printable characters rendered as dots between messages are the framing itself — the four-byte length prefix and the one-byte record type — so the capture confirms both that the content is exposed and that the framing behaves as specified.

Anyone with access to the network segment reads the entire conversation, including the usernames. This is the baseline that the next phase must destroy: repeating this identical capture after adding confidentiality must show unreadable data at the same offsets.

3.5 Evidence index

All files are in `evidence/phase1/`.

File	Contents
<code>server.pcap</code>	Capture on <code>vm1-server</code> (54 packets, both client connections)
<code>c1.pcap</code>	Capture on <code>vm2-client1</code> (27 packets)
<code>c2.pcap</code>	Capture on <code>vm3-client2</code> (27 packets)
<code>server.log</code>	Server relay log with full message content
<code>wireshark-packet-list.png</code>	Wireshark packet view, plaintext in the bytes pane
<code>wireshark-follow-tcp-stream.png</code>	Follow → TCP Stream, whole conversation

3.6 Deliverables checklist

Requirement	Status
One server and two clients connected simultaneously over TCP	Done
Messages delivered $C1 \rightarrow C2$ and $C2 \rightarrow C1$ via the server	Done
Server relays each message as-is, without encryption	Done
Command interface <code>@user</code> , <code>/chat</code> , <code>/who</code> , <code>/quit</code>	Done
Server logs every relayed message; can read full content	Done
Wireshark capture with Follow → TCP Stream showing plaintext	Done
Protocol description: framing, message completeness, routing	Done

4 Phase 2 — Confidentiality via Diffie–Hellman

What changed since Phase 1. A Diffie–Hellman handshake now runs before any application data, and every record after it is encrypted with AES-256-GCM; the login itself travels encrypted. The record framing, the LOGIN/MSG/WHO grammar, and the server’s routing are *unchanged* — the server simply decrypts inbound records and re-encrypts outbound ones. A man-in-the-middle proxy is added to attack the result.

4.1 Requirements

Phase 2 requires a Diffie–Hellman key exchange implemented from first principles (a standard published group, with the modular exponentiation written by hand — no library DH), an AES key derived from the shared secret by hashing rather than using it raw, authenticated encryption on all traffic, a printed fingerprint of the secret on both ends, and a demonstration that the traffic is

no longer readable and that tampering is detected. It also requires a separate man-in-the-middle proxy that performs two independent exchanges and reads everything.

4.2 Diffie–Hellman from first principles

The group is RFC 3526 group 14: a standard, published 2048-bit MODP group with generator $g = 2$, whose modulus is a safe prime (both p and $(p - 1)/2$ are prime). Each side picks a 256-bit private exponent x and sends $g^x \bmod p$; each then computes the shared secret $Z = (\text{peer})^x \bmod p$.

The modular exponentiation is written by hand as *square-and-multiply*, reducing modulo p at every step. Reducing at each step is not an optimisation but a necessity: with a 2048-bit modulus and a 256-bit exponent, computing g^x before reducing would require an integer with more digits than there are atoms in the observable universe. OpenSSL’s big-integer type is used only as an arithmetic primitive (BN_mod_mul); BN_mod_exp and the entire DH API are deliberately avoided, as the assignment requires. A received public value is rejected unless $1 < y < p - 1$, which prevents a peer from forcing the secret to a value it chooses. A self-test (selftest.txt) confirms the group is a 2048-bit safe prime and that the hand-written exponentiation agrees with OpenSSL’s reference on a range of inputs.

4.3 Why the shared secret is hashed

The raw secret Z is *not* used as the AES key. Z is a 2048-bit element of a mathematical group: its size is wrong for AES-256 (which needs exactly 32 bytes), and its bits are not uniformly distributed — some values are structurally more likely, and there are known relationships among the bits of a modular-exponentiation result. Keying AES with such biased material is unsound. A cryptographic hash fixes both problems at once: SHA-256 outputs exactly 32 bytes and maps the biased input to an output indistinguishable from random. Hashing also lets a label and the handshake transcript be folded in, so the single secret yields several independent values that cannot be confused with or derived from one another:

```
K_c2s = SHA256("CS6008-P2-KEY|c2s|" || Z || TH),    K_s2c = SHA256("...|s2c|" ...),    FP
          = SHA256("CS6008-P2-FP|" ...) [0..7]
```

The two directional keys prevent a record the server sent from being replayed back at it as though the client had sent it. The fingerprint FP uses a *different* label from the keys, so it can be printed safely; the raw secret and the keys themselves are never printed.

4.4 Record encryption

After the handshake, each REC_APPDATA body is ciphertext||tag under AES-256-GCM. Neither an IV nor a sequence number is transmitted: the nonce is salt(4)||seq(8) where seq is an implicit per-direction counter each side maintains independently. TCP delivers records in order, so the counters stay in lockstep; any drop, duplication, reorder, or injection desynchronises them, the tag fails, and the connection aborts. A monotonic counter also guarantees the nonce never repeats under one key, which GCM requires.

4.5 Verification

Matching fingerprints, secret never printed. Each client prints the fingerprint of its derived secret, and it equals the one the server logs for that connection; the two clients differ, as each runs its own exchange (chat-transcript.md):

```
alice(C1) | key exchange complete, shared-secret fingerprint f5b73958da76b903
SERVER    | KEX ... fingerprint=f5b73958da76b903      <- matches alice
bob(C2)   | key exchange complete, shared-secret fingerprint ef643199d65ce813
SERVER    | KEX ... fingerprint=ef643199d65ce813      <- matches bob
```

Traffic is unreadable. The Phase 1 capture was repeated on the encrypted protocol (`chat.pcap`). Every chat word from the session was searched across all frames and found in none. Figure 4 shows a chat record’s bytes as AES-GCM ciphertext where Phase 1 (Figure 2) showed readable ASCII; Figure 5 shows Follow → TCP Stream unreadable where Phase 1 (Figure 3) showed the whole conversation.

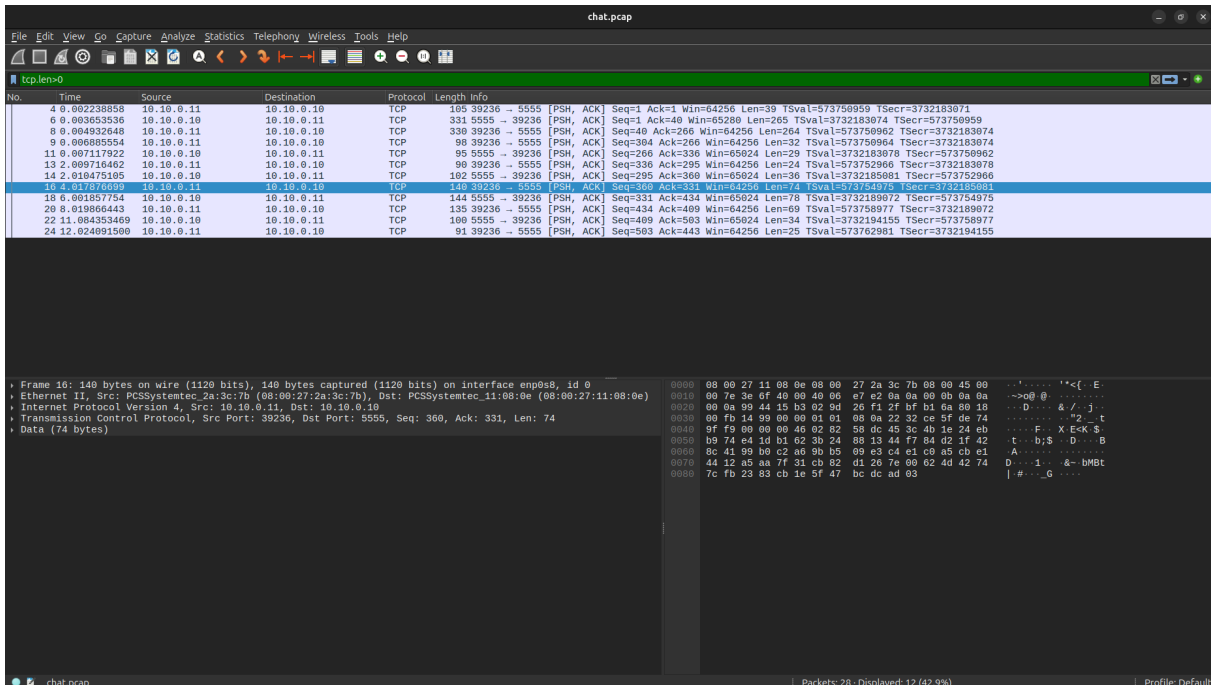


Figure 4. Phase 2, `chat.pcap`, frame 16 (a chat record). The bytes pane is AES-GCM ciphertext with an unreadable ASCII column — contrast Figure 2, the same view in Phase 1 showing `MSG bob hey bob, are you there?` in cleartext.

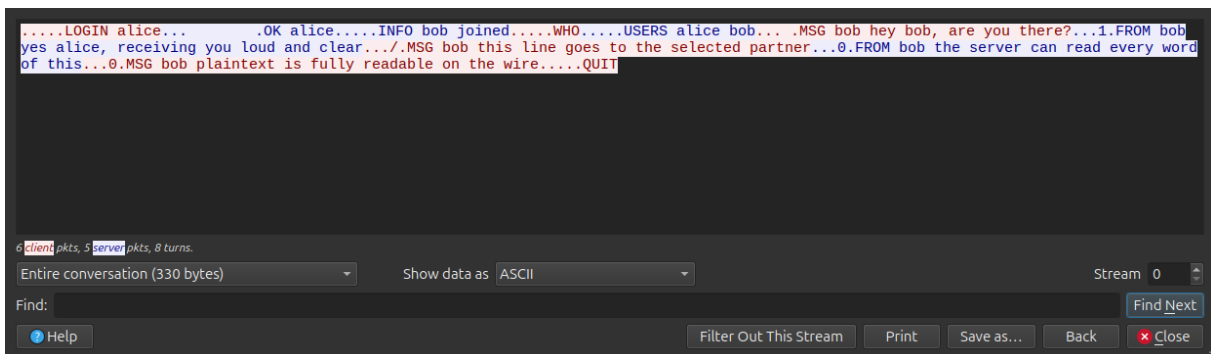


Figure 5. Phase 2, Follow → TCP Stream. Client data in red, server in blue — all unreadable, except the two Diffie–Hellman public values (which are meant to be public). Contrast Figure 3, the same view in Phase 1 with every line legible.

Tampering is rejected. With the proxy’s `--tamper` flag flipping one byte of a chat record, the login (untampered) is accepted but the corrupted record fails its GCM tag; the server logs the message AES-GCM authentication failed and closes the connection (`tamper-transcript.md`). A single flipped byte is a hard failure, never silent corruption.

4.6 Attack — man-in-the-middle

Diffie–Hellman agrees a key with whoever answers the connection; it never checks who that is. The proxy (`mitm.cpp`) exploits this: pointed at by the victim, it runs one exchange as the server toward the victim and a second as a client toward the real server, holding both keys.

```
victim <-- DH: session V --> MALLORY <-- DH: session S --> real server
```

With `alice` pointed at the proxy (`./client 10.10.0.13 alice`) and `bob` at the real server, Mallory read and logged every message in plaintext while `alice`'s client reported a normal, completed, encrypted session (`mitm-transcript.md`):

```
MALLORY | two sessions established -- victim-side fp=cfb2a3e... server-side fp=6d7bdab0...
MALLORY | [C->S] MSG bob this channel feels private to us
MALLORY | [C->S] MSG bob mallory is reading every word
```

Detectable evidence, and why a user misses it. The evidence is the fingerprints. `Alice` computed `cfbb2a3e...` for her session with Mallory, while the real server computed `6d7bdab0...` for its session with Mallory — two different secrets. Had `alice` and `bob` compared fingerprints over an independent channel, the mismatch would have exposed that neither was talking directly to the other. An ordinary user misses this because unauthenticated Diffie–Hellman gives no automatic signal: both ends see a normal handshake and a working encrypted session, and the only tell is a manual, out-of-band comparison that users never perform. This is exactly the gap Phase 3 closes.

4.7 Deliverables checklist

Requirement	Status
DH from scratch, standard published group, hand-written modular exponentiation	Done
AES key derived from the DH secret by hashing, not used raw	Done
Authenticated encryption (AES-GCM) on all traffic, login included	Done
Both ends print matching fingerprints; raw secret/key never printed	Done
Wireshark shows the traffic is no longer readable (before/after)	Done
Tampering with a ciphertext byte is rejected, not silently accepted	Done
MITM proxy with two independent DH exchanges reads all traffic	Done
Explanation: detectable evidence of the MITM and why a user misses it	Done

5 Phase 3 — Server Authentication via PKI

What changed since Phase 2. Before the key exchange, the server now sends a CA-signed certificate and a signature proving it holds the matching private key; the client validates both and aborts if anything is wrong. The record encryption, application grammar and routing are unchanged. The same man-in-the-middle proxy that succeeded in Phase 2 now fails.

5.1 Requirements

Phase 3 requires a Certificate Authority and a CA-signed server certificate (via OpenSSL); the server sending its certificate before the DH exchange; the client validating the certificate's chain, validity period and identity, and aborting on failure without revealing anything further; the server proving possession of its private key; and a re-run of the Phase 2 MITM showing it now fails, including the case where the attacker holds the certificate file but not its private key.

5.2 The PKI

Three OpenSSL scripts (phase3/pki/) build the infrastructure on the server VM: a CA (private key and self-signed root), then a server key and CSR that the CA signs. The certificate binds the name `chatserver.local` (and IP:10.10.0.10) via Subject Alternative Names.

```
issuer= CN=CS6008 Root CA
subject= CN=chatserver.local
X509v3 Subject Alternative Name: DNS:chatserver.local, IP Address:10.10.0.10
$ openssl verify -CAfile ca-cert.pem server-cert.pem -> server-cert.pem: OK
```

5.3 Certificate and proof of possession

The handshake gains two server messages: `HS_SERVER_CERT` (before the DH value) and `HS_SERVER_PROOF` (after it). The proof is

```
sig = RSA-SHA256_sign( server_key, version || client_random || cert || server_pub )
```

This distinction is the heart of the phase. A *certificate* only asserts that a CA once vouched that a public key belongs to `chatserver.local`; it is a public file, and anyone can copy it. It does not prove that the party presenting it actually holds the corresponding private key. The *proof of possession* closes that gap: only the holder of the private key can produce a signature that verifies against the public key in the certificate. Including `client_random` in the signed value binds the proof to this session, so a signature captured from one session cannot be replayed into another. The certificate answers “whose key is this?”; the signature answers “does this peer hold it, now?”.

5.4 Client validation

The client checks the following in order and aborts at the first failure, sending an alert and nothing else — crucially, it has not yet sent its own DH value, so a client that rejects the server has revealed only 32 random bytes:

1. **Chain** — the certificate verifies against the trusted CA (`X509_verify_cert`).
2. **Validity** — the current time is within the certificate’s window.
3. **Identity** — a SAN/CN matches the expected name (`X509_check_host`).
4. **Proof of possession** — the signature verifies against the certificate’s public key.

The `certcheck` tool runs this exact logic over each certificate variant (`cert-validation.txt`); every bad certificate is rejected by precisely the check that should catch it:

```
ACCEPT server-cert.pem : ok
REJECT mallory-cert.pem : not signed by the trusted CA (a) chain
REJECT expired-cert.pem : outside its validity period (b) validity
REJECT wrongname-cert.pem : identity does not match expected server (c) identity
```

5.5 Verification — legitimate flow

Both clients validate the certificate and complete the exchange (`chat-transcript.md`):

```
alice(C1) | server certificate verified (chatserver.local); key exchange complete, fingerprint 6069079c...
bob(C2)   | server certificate verified (chatserver.local); key exchange complete, fingerprint ...
```

The certificate travels in the clear in the handshake — which is correct, a certificate is public. Figure 6 shows it in the capture, with CS6008 Secure Chat, CS6008 Root CA and `chatserver.local` readable in the certificate record, while no chat word appears anywhere in the capture. Figure 7 shows the same in Follow → TCP Stream: the certificate is legible, everything after it is ciphertext.

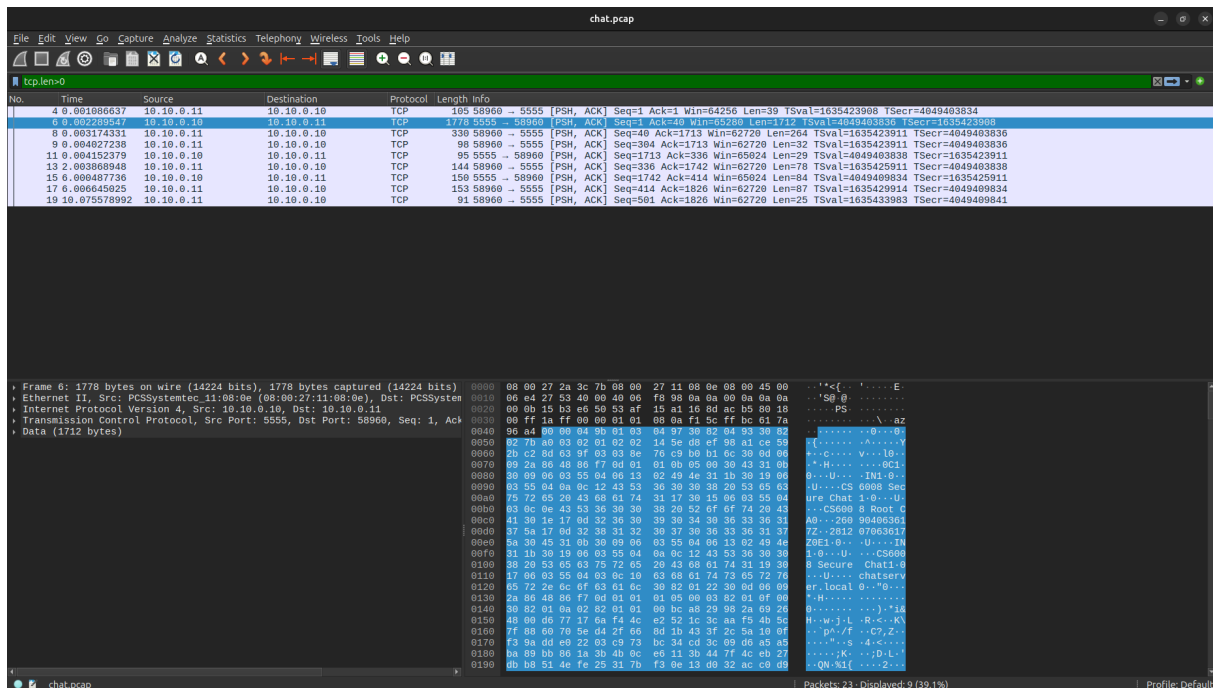


Figure 6. Phase 3, chat.pcap, frame 6 — the server’s certificate in the handshake. The ASCII column shows the readable X.509 identity (CS6008 Secure Chat, Root CA, chatserver.local). A certificate is public, so this is expected; the chat content that follows remains AES-GCM encrypted.

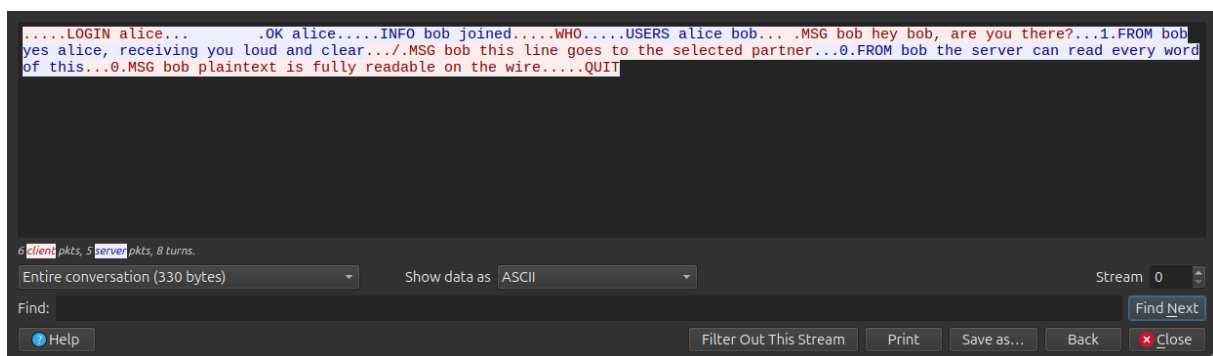


Figure 7. Phase 3, Follow → TCP Stream. The server’s certificate is readable in the handshake (blue); the Diffie–Hellman values and all chat that follow are unreadable ciphertext.

5.6 Attack — the MITM now fails

The Phase 2 proxy is re-run against the authenticated protocol. It cannot obtain a CA-signed certificate, so it is rejected two ways.

Self-signed certificate. Mallory presents a self-signed certificate with the right name. The chain check fails and the client aborts before sending anything (`mitm-selfsigned-transcript.md`):

```
MALLORY | victim rejected the presented certificate -- attack failed
alice(C1) | handshake aborted: certificate rejected: not signed by the trusted CA
```

Stolen certificate, no private key (§4.2). Mallory copies the real `server-cert.pem` (a public file) but lacks its private key, so she signs with a key of her own. The certificate passes chain, validity and identity, but the proof of possession fails (`mitm-stolen-transcript.md`):

```
alice(C1) | handshake aborted: proof of possession failed:
          server does not hold the certificate's private key
```

In both cases the client reveals nothing — a direct contrast to Phase 2, where this same proxy read every message.

5.7 Deliverables checklist

Requirement	Status
CA and CA-signed server certificate via OpenSSL	Done
Server sends its certificate before the DH exchange	Done
Client validates chain, validity period and identity	Done
On failure: aborts, sends no password / no DH / nothing further	Done
Server proves possession of the private key	Done
Full legitimate flow: validate, complete DH, chat	Done
Same MITM re-run now fails (self-signed cert)	Done
Attacker with the cert file but not the key is rejected (proof of possession)	Done

6 Phase 4 — End-to-End Encryption Between Clients

What changed since Phase 3. The two clients now establish a shared key directly, and encrypt chat to each other *inside* the existing client-server link, so the server relays only opaque data it cannot read. The entire feature is in the client: `server.cpp` is byte-for-byte identical to Phase 3.

6.1 Requirements

Phase 4 requires the two clients to establish a shared secret that is independent of, and not derivable by, the server, using the server purely as a relay; triggered by `/e2e username` and carried with the fixed wire tags `__E2E_INIT__`, `__E2E_ACK__` and `__E2E_MSG__`, so the server's routing need not change; with the server seeing only opaque data for chat once the session is established; independently-computed fingerprints on both clients that match; and the inner layer sitting inside the Phase 3 encrypted link.

6.2 Design

The E2E layer lives entirely inside the `<text>` field of an ordinary MSG/FROM message, base64-encoded because that field is text. `/e2e bob` runs a second Diffie–Hellman, over the same MODP-2048 group, directly between the clients:

```
alice -> server -> bob    MSG bob    __E2E_INIT__<b64(ver, epoch 0, A)>
bob   -> server -> alice  MSG alice  __E2E_ACK__ <b64(ver, epoch 0, B)>
both derive Z_e2e = peer^own mod p, derive keys, print FP_e2e
```

The keys are labelled by direction — “a” is the lexicographically smaller username, “b” the larger — so both ends agree on which key is which without negotiating. The payload carries an epoch field, always 0 here, so that Phase 5’s rekeying needs no format change.

The server does not change. Because the tags ride inside `<text>`, which the server copies verbatim, `server.cpp` is unmodified from Phase 3 — a fact confirmed by `diff`, and visible in the evidence: the Phase 4 server’s start banner still reads `START phase3`, because it is the same binary. This is assignment §1.4 property (1) satisfied by construction.

Two layers. An E2E message is encrypted with the $C1 \leftrightarrow C2$ key, then the whole line is encrypted again by the client-server link on the way to the server. The server can decrypt the outer layer — so it logs the message — but sees only the opaque `__E2E_MSG__` base64 for chat content; a wire sniffer sees neither layer.

Dispatch (§1.4 property 2). Each incoming FROM is dispatched on its tag: handshake tags complete the session and are never shown as chat; a `__E2E_MSG__` is decrypted and shown as sender `[e2e]>`; anything else is plain chat. Outgoing text starting with `__E2E` is refused (a user cannot forge a tag), and a plain message from a peer with an active E2E session is flagged as a downgrade rather than shown normally.

6.3 Verification

Both clients print a matching end-to-end fingerprint, independent of the separate fingerprints of their client-server sessions (`e2e-transcript.md`):

```
alice(C1) | server session fingerprint 26ea5ff5...  bob(C2) | server session fingerprint 2138f6c1...
alice(C1) | end-to-end established with bob,   fingerprint 1cf3a11922709169
bob(C2)   | end-to-end established with alice, fingerprint 1cf3a11922709169    <- matches
bob(C2)   | alice [e2e]> now the server sees only ciphertext for our messages  <- decrypted by bob
```

The server’s relay log shows the required before/after contrast (`server-view.txt`): a message sent before `/e2e` is logged in full, but every message after is opaque.

```
before /e2e: RELAY alice -> bob : "this line is only server-encrypted, the server can read it"
after  /e2e: RELAY alice -> bob : "__E2E_MSG__AAAAAKHFPIs0SpKJ+y00bwVM0bvJSfJSx50DMs9Jz/+01Y4..."
```

On the wire the content is doubly hidden: searching the capture for the chat words and even the E2E tags found them all zero, except `chatserver.local` once (the certificate). Figure 8 shows the Follow → TCP Stream: the certificate is readable in the handshake, everything after is ciphertext.

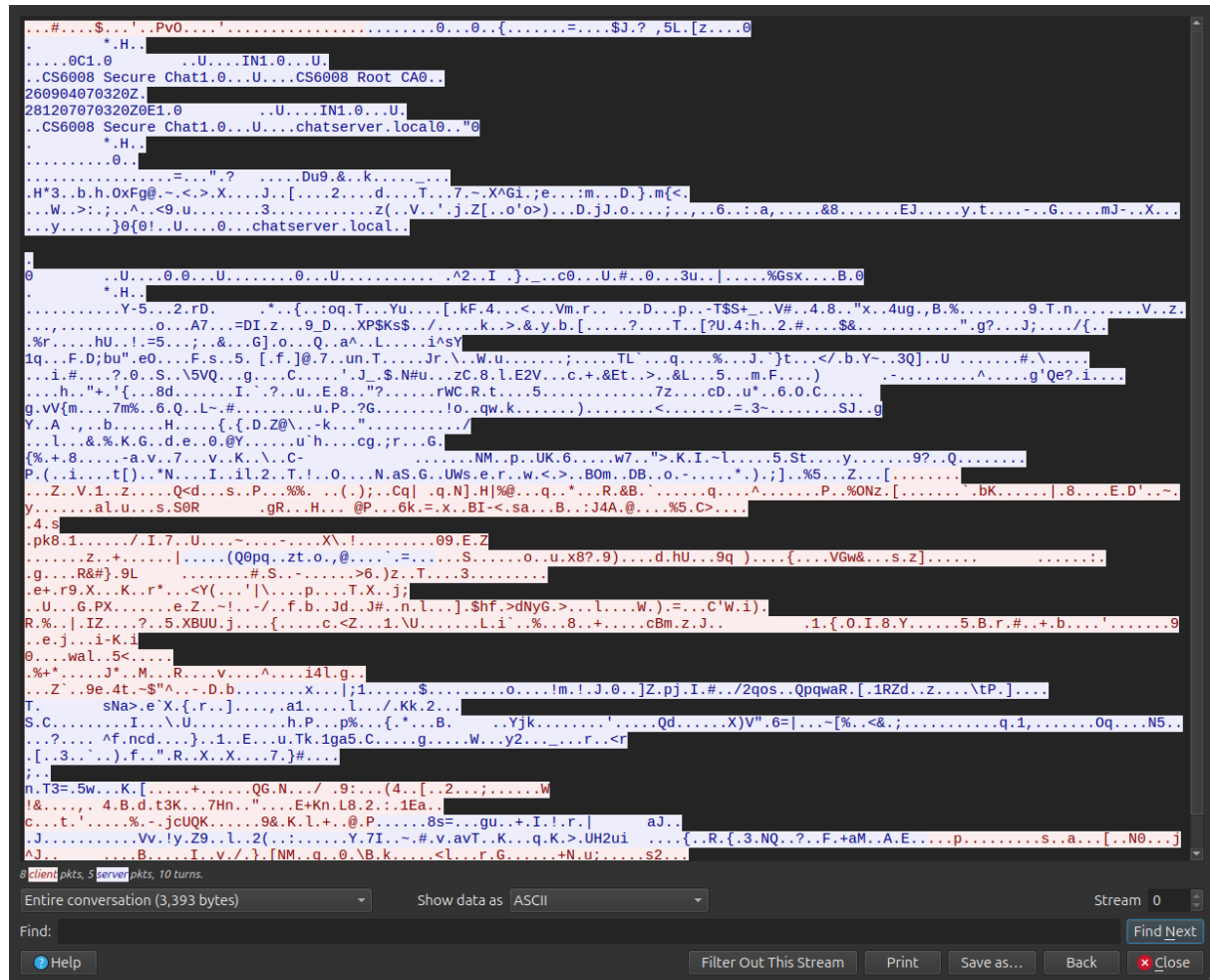


Figure 8. Phase 4, Follow $\rightarrow$ TCP Stream. Only the certificate in the handshake is readable; all chat — including the E2E tags, which sit inside the outer encryption — is ciphertext. The E2E content is hidden from a wire sniffer, and (per the server log) from the relay itself.

6.4 Deliverables checklist

Requirement	Status
C1↔C2 shared secret, independent of and not derivable by the server	Done
Triggered by /e2e, using the fixed <code>__E2E_*</code> tags	Done
Server routing unchanged (<code>server.cpp</code> identical to Phase 3)	Done
Server sees only opaque data for E2E chat; contrast with a pre-E2E message	Done
Independently-computed fingerprints on both clients match	Done
A C1 message is correctly decrypted and displayed by C2	Done
E2E is an inner layer, inside the Phase 3 client-server link	Done
Handshake tags never shown as chat; chat never treated as handshake	Done

7 Phase 5 — Forward Secrecy via Key Rotation

What changed since Phase 4. The end-to-end key is now renegotiated every 60 s. Each new key comes from a fresh Diffie–Hellman, independent of the last, and the old key is discarded — so a compromised key exposes only a bounded window of traffic. There is no wire-format change (the epoch field already existed), and `server.cpp` is still identical to Phase 3.

7.1 Requirements

Phase 5 requires the end-to-end key to be renegotiated on a fixed timer while the session is active, each new key independent of the old and the old key discarded; simultaneous rotation attempts handled without the two sides diverging; chat uninterrupted across a rotation; at least two rotations logged with timestamps and fingerprints that change each time and match on both sides; and an explanation of what forward secrecy adds over Phase 4.

7.2 Design

A rotation is simply another `__E2E_INIT__` / `__E2E_ACK__` exchange for the next epoch, so no wire format changes. Each epoch’s key is a fresh ephemeral Diffie–Hellman and is never derived from the previous key; the retiring key lingers for a short grace window (decrypt-only, ~10 s) so a message in flight across a rotation still decrypts, and is then zeroised. Because every `__E2E_MSG__` carries its epoch, the recipient always knows which key to use.

Collision avoidance. Only the *initiator* — the peer with the lexicographically smaller username — starts a rotation, at 60 s; both sides compute the same initiator with no negotiation, since usernames are unique. The other peer holds a 75 s fallback and initiates only if the initiator has gone silent. Should both send an `__E2E_INIT__` for the same epoch at once, the smaller username wins and the loser answers with an `__E2E_ACK__`. This is deterministic, so the two sides can never settle on different keys — confirmed by the self-test’s “both sides agree on the fingerprint after each rotation”.

7.3 Verification

Both clients log each epoch’s fingerprint with a timestamp (`fingerprint-timeline.md`). The interval was set to 20 s via `--rekey 20` so two rotations complete within a short captured session; the default is 60 s.

epoch	time (alice)	alice fingerprint	bob fingerprint	match
0	07:22:40	edadeade4196261a	edadeade4196261a	yes
1	07:23:01	d5b9d6fbd2bad80f	d5b9d6fbd2bad80f	yes

```
2      07:23:21      0ab850ced4712d7f      0ab850ced4712d7f      yes
```

Two rotations: the fingerprint changes at each, and both clients agree after each rotation completes. Chat is not disrupted — from `e2e-rekey-transcript.md`, messages sent before, across, and after each rotation are all delivered:

```
07:22:43  alice -> bob [e2e]: epoch 0: forward secrecy demo      (epoch 0, decrypted by bob)
07:23:01  E2E rekey -> epoch 1 ...
07:23:01  alice -> bob [e2e]: after the first rotation          (epoch 1, decrypted by bob)
07:23:03  bob -> alice [e2e]: replying across a rekey           (epoch 1, decrypted by alice)
07:23:21  E2E rekey -> epoch 2 ...
07:23:21  alice -> bob [e2e]: after the second rotation         (epoch 2, decrypted by bob)
```

The message at 07:23:21 is sent in the same second as the rotation and decrypts correctly. The server's relay log stays opaque throughout, and the epoch counter is even visible in the base64 (`__E2E_INIT__` payloads begin `BQAAAAAB`, `BQAAAAEB`, `BQAAAAIB` for epochs 0, 1, 2).

7.4 What forward secrecy buys

In Phase 4 a single end-to-end key protects the whole session. An attacker who records the ciphertext and *later* obtains that key — by stealing it from a device after the fact, or through a future cryptographic break — can decrypt *every* message ever sent in that session. Phase 5 rotates the key every 60 s, each epoch keyed by a fresh ephemeral Diffie–Hellman whose private exponents are discarded after the exchange, and wipes the old key. Compromising one epoch's key now reveals only that ~60 s window: earlier and later epochs used independent secrets that cannot be reconstructed from the leaked key. That is forward secrecy — the guarantee that compromising a key *now* does not retroactively expose traffic already protected by discarded keys. Phase 4 alone gives no such guarantee.

7.5 Deliverables checklist

Requirement	Status
E2E key renegotiated on a fixed timer while the session is active	Done
Each new key independent of the old; old key discarded	Done
Simultaneous-rotation collisions handled without divergence	Done
Chat uninterrupted across a rotation	Done
≥2 rotations logged with timestamps + fingerprints, changing and matching	Done
A message sent immediately after a rotation is decrypted by the recipient	Done
Explanation of what forward secrecy adds over Phase 4	Done

8 Building and Running

```
cd phaseN && make          # phase2: + mitm, selftest.  phase3: + certcheck
make test                  # run the crypto self-test (phase 2+)

# phase 3: build the PKI first (on the server VM, the CA host)
cd pki && ./make-ca.sh && ./make-server-cert.sh && ./make-attack-certs.sh

# server (10.10.0.10) -- phase 3 needs --cert/--key
./server                  # phase 1-2
./server --cert pki/out/server-cert.pem --key pki/out/server-key.pem # phase 3

# clients (10.10.0.11 / .12) -- phase 3 needs --ca
./client 10.10.0.10 alice # phase 1-2
./client 10.10.0.10 alice --ca certs/ca-cert.pem --expect chatserver.local # phase 3-4
# phase 4-5: type "/e2e bob" to start end-to-end; the key rotates every 60 s
# (add --rekey 20 to both clients to see rotations sooner)
```

```
# MITM on vm4-mallory, then point the victim at 10.10.0.13
./mitm --listen 10.10.0.13 --server 10.10.0.10 --log mallory.log [--tamper] # phase 2
./mitm ... --ca certs/ca-cert.pem --present-cert C --present-key K      # phase 3

# packet capture, on any VM
sudo tshark -i enp0s8 -f 'tcp port 5555' -w /tmp/cap.pcap
```

Two environment notes recorded while capturing: `dumpcap` drops privileges even when invoked through `sudo`, so `tshark -w` cannot write into a home directory and must target a world-writable path; and Ubuntu’s AppArmor profile prevents `tshark -r` from reading capture files stored under `/home`.

A Generative AI Usage

As required by assignment section 1.5, every prompt used during this assignment is recorded in

`docs/ai-prompts.md`

in the submitted repository. The file is maintained incrementally alongside the work rather than reconstructed afterwards: entries are added in the commit that contains the work they relate to, so the git history shows the prompts accumulating phase by phase. It is organised by phase, and each entry records the prompt together with what came out of it.

It covers Phase 0 (lab infrastructure, SSH access, network configuration, tooling and documentation) and Phases 1 through 5 (protocol design, implementation, evidence collection and this report).